KITE: A Kinematics-Informed Trajectory Estimator for 3D UAV Forecasting

Anonymous Author(s)

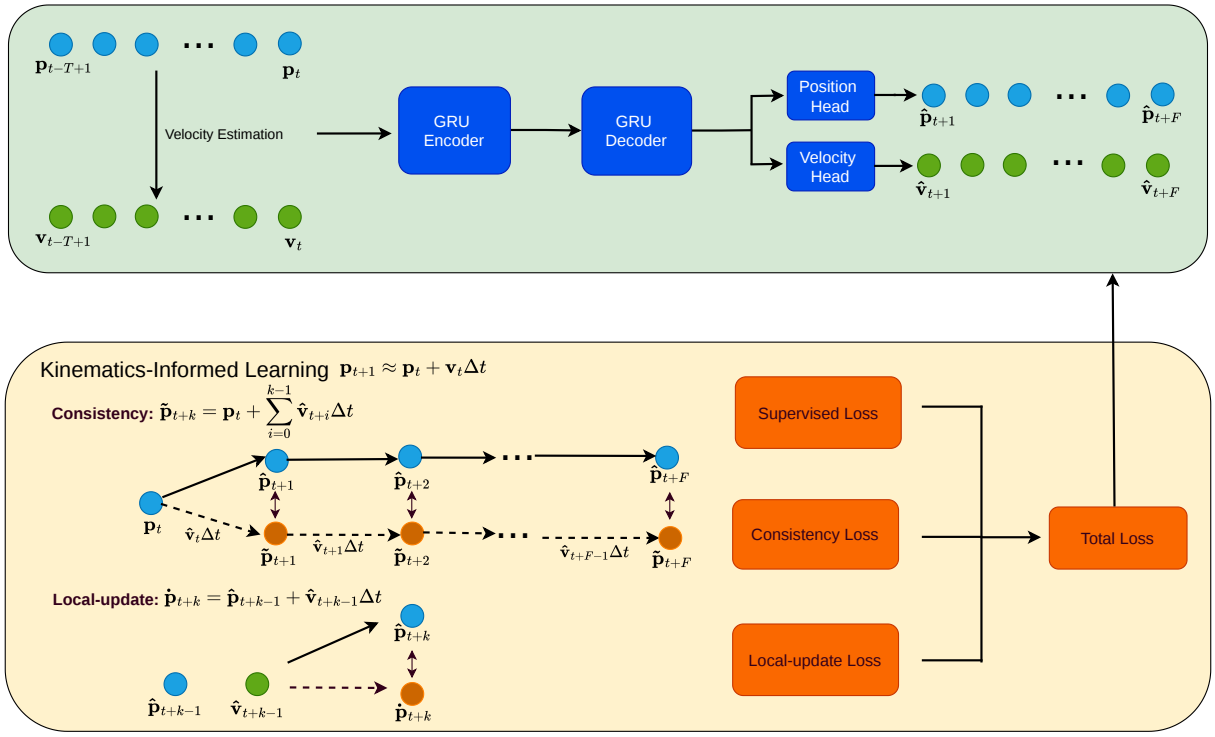


Figure 1: KITE Architecture

Abstract

Unmanned Aerial Vehicle (UAV) trajectory forecasting in three-dimensional space is essential for trajectory planning and safe navigation, but remains challenging due to noisy measurements, complex flight dynamics, and the need for long-horizon stability. In this work, we propose KITE, a lightweight kinematics-informed GRU model for robust trajectory prediction. KITE augments positional inputs with Savitzky–Golay-filtered velocity estimates and is trained using a hybrid objective that combines standard supervised losses with physics-guided constraints enforcing discrete-time kinematic consistency between position and velocity. Extensive experiments on a recent mixed synthetic–real UAV trajectory dataset demonstrate that KITE achieves the lowest average prediction error and the longest valid prediction time among a wide range of classical and state-of-the-art baselines. These results indicate that incorporating lightweight kinematic structure into recurrent sequence models significantly improves both prediction accuracy and long-horizon stability, producing physically plausible trajectory forecasts suitable for real-world UAV navigation tasks.

CCS Concepts

• **Computing methodologies** → **Neural networks**.

Keywords

UAV trajectory forecasting, Kinematics-informed, GRU

ACM Reference Format:

Anonymous Author(s). 2026. KITE: A Kinematics-Informed Trajectory Estimator for 3D UAV Forecasting. In *Proceedings of November 07–11, 2026 (CIKM '26).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '26,

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Accurate forecasting of a single Unmanned Aerial Vehicle (UAV) trajectory in three-dimensional space is essential for safe navigation and reliable trajectory planning in dynamically changing environments. In practice, a predictor must handle noisy pose measurements, diverse flight patterns, and long forecasting horizons while keeping the predicted path physically plausible.

Traditional recurrent models excel at capturing short-term motion patterns but often drift in accuracy over extended prediction horizons. Transformer-based and continuous-depth models, such as NeuralODE models, boost expressiveness yet introduce instability and heavy computational demands that hinder real-time deployment. Moreover, purely data-driven methods overlook fundamental kinematic relationships–like the discrete-time linkage between position and velocity–that could serve as a powerful inductive bias for trajectory prediction.

We introduce KITE, a kinematics-informed trajectory estimator for multi-step 3D UAV trajectory forecasting. KITE augments positional inputs with velocities estimated via a Savitzky–Golay differentiation filter, producing a denoised 6D state. It then uses dual decoder heads to jointly predict future positions and velocities with a hybrid loss that combines supervised errors with two physics-guided consistency terms derived from discrete-time kinematics. Evaluated on the mixed synthetic–real UAV dataset [9], our model outperforms strong recurrent, transformer, and trajectory-specialized baselines in ADE, RMSE, and valid time, demonstrating that lightweight kinematic guidance substantially improves long-horizon 3D UAV trajectory prediction.

2 Related Work

2.1 Conventional Mathematical Models

Traditional approaches to UAV trajectory forecasting mainly rely on explicit mathematical descriptions of motion, including kinematic models, dynamic models, Kalman filtering formulations, graph-based formulations, and aerodynamic models [12]. These methods remain attractive because they provide a transparent description of UAV motion, making them highly interpretable; they are relatively inexpensive to run compared to most machine learning models; and they can be readily integrated into existing control pipelines. In many practical systems, especially when the computational resources are limited or when domain knowledge about UAV dynamics is available, these analytical models provide a principled and efficient starting point for trajectory forecasting.

Among these mathematical models, state-space filtering approaches that combine noisy observations with motion priors are widely explored. Kalman filters have long been used for UAV state estimation because they support recursive real-time updates and can incorporate prior assumptions about velocity, acceleration, and maneuver modes. More advanced extensions, such as interacting multiple-model (IMM) filters, attempt to handle switching motion patterns by combining several candidate dynamic models. These approaches are useful in tracking settings, where the goal is to denoise measurements and extrapolate motion over a short horizon [2, 14, 20].

Another popular direction formulates UAV trajectory prediction or planning as an optimization problem under motion and mission constraints. In these methods, motion is typically governed by explicit kinematic or aerodynamic equations, while objectives may include minimizing energy use, satisfying communication requirements, ensuring collision avoidance, or maintaining dynamically feasible flight paths. Graph-based and path-search formulations similarly discretize the state space or feasible routes and compute future motion through shortest-path or constrained search strategies, and such methods are relatively effective when the environment and system dynamics are sufficiently structured [1].

The main limitation of mathematical models is that their accuracy depends heavily on the validity of the assumed model. UAV motion in practice is influenced by sensor noise or environmental uncertainty that are difficult to capture with simple closed-form models alone. Although conventional methods offer interpretability and efficiency, their performance tends to degrade when the flight regime differs from the assumptions of the underlying model. This limitation motivated the use of machine learning approaches, in which motion patterns are learned directly from data rather than specified by hand-crafted equations [12].

2.2 Machine Learning Models

To overcome the limitations of conventional mathematical models, later work increasingly turned to machine learning methods that learn to predict future trajectories directly from historical data. Early statistical and classical machine learning approaches modeled future positions using regression and probabilistic estimation, offering greater flexibility compared to conventional models. However, deep learning, especially sequence models, is the one that really transformed learning-based UAV trajectory forecasting.

Recurrent neural networks (RNNs), especially Long Short-term Memory Networks (LSTMs) or Gated Recurrent Units (GRUs), are strong baseline models for UAV trajectory prediction because they can process historical data and learn temporal dependencies over multiple time steps. A representative example is the recurrent LSTM-based UAV trajectory prediction framework using ADS-B data, which showed that sequential neural models can be used to forecast future UAV states from historical flight traces. Related work on aircraft trajectory prediction under ADS-B failure also used Bi-LSTM architectures to improve short-term trajectory prediction and enhance robustness in degraded sensing conditions. These models demonstrate the potential of recurrent learning for air-motion forecasting, where they avoid explicit hand-crafted equations and instead extract motion patterns from data, which can improve performance in scenarios where the true system behavior is too complex to model analytically [13, 15, 17].

Besides RNNs, learning-based trajectory prediction has also expanded toward higher-capacity architectures that can capture longer-range dependencies and more complex motion distributions [7]. Transformer-based approaches have become more common in trajectory prediction research because attention mechanisms are well-suited to modeling long temporal dependencies and interactions among multiple features or agents. Recent 3D trajectory prediction work, such as 3DMoTraj [18], follows this trend by using attention to explicitly model dependencies across spatial axes and refine 3D motion forecasts.

Despite their flexibility, purely data-driven models still have limitations. They typically require large training data and can suffer from degraded generalization when deployment conditions differ from the training environment. This issue happens in UAV applications, where trajectory statistics may change across mission types or environments. Additionally, positional data alone can be limiting for UAV forecasting, because models trained only to minimize coordinate prediction error may produce trajectories that match the ground truth locally but violate basic motion relationships over longer rollouts. This has motivated physics-informed approaches that retain the flexibility of machine-learning-based predictors while ensuring that the resulting trajectories remain physically plausible over long horizons [17, 18].

2.3 Physics-informed Models

Beyond purely data-driven forecasting, recent work has started to incorporate physical knowledge about UAV motion into learning models to improve long-horizon plausibility. One direction focuses on physics-informed neural networks (PINNs) that encode the governing dynamics directly in the training objective. In this setting, the neural network approximates the dynamics while residuals of the underlying equations of motion appear as penalty terms in the loss. For instance, PINNs have been used to learn quadrotor dynamics by embedding the known equations of motion in the training process, so that the learned model remains consistent with quadrotor physics even in the presence of non-conservative aerodynamic effects. Related approaches employ PINNs to generate dynamically feasible UAV paths, where the network minimizes both trajectory-tracking error and residuals of the dynamic constraints, yielding collision-free trajectories without requiring large supervised datasets of optimal paths. These methods provide strong physical guarantees but rely on detailed dynamic models, which limit their applicability when the underlying dynamics are only partially known [5, 11].

Another direction focuses on kinematics-informed sequence models that keep the simple architecture of recurrent predictors while incorporating basic first-order motion structure into the model, such as velocity. Recent velocity-aware architectures such as VECTOR exemplify this idea [9]. Instead of treating future positions as the sole outputs of a position-only sequence, VECTOR uses historical velocity estimates as its primary input and jointly predicts future velocities and positions. The authors argue that incorporating first-order dynamics through velocity measurements allows the model to better capture UAV motion and improves prediction accuracy and generalization compared to position-only inputs.

KITE follows this kinematics-informed direction but goes beyond using velocity only as an input signal. Instead of relying solely on velocity-based representations, KITE treats kinematic relationships as soft constraints in the training objective. In this way, it bridges PINN-inspired physics guidance and recurrent kinematics-informed models, aiming to preserve the flexibility and simplicity of recurrent models while improving the physical plausibility of long-horizon UAV trajectory predictions.

3 Methodology

We consider forecasting UAV trajectories in three-dimensional space, where each position consists of longitude, latitude, and altitude coordinates. At each discrete time step t, with sampling interval Δt, the UAV position is represented as $\mathbf{p}_t \in \mathbb{R}^3$. Given a sequence of T historical positions $\{\mathbf{p}_k\}_{k=t-T+1}^{t}$, the objective is to predict the next F future positions $\{\hat{\mathbf{p}}_{t+k}\}_{k=1}^{F}$.

During training, trajectory data are segmented into windows of length $T+F$. Without overlap, a trajectory of length N creates $\left\lfloor \frac{N}{T+F} \right\rfloor$ training samples of length $T+F$. By introducing a stride $S \ll (T+F)$, we construct overlapping windows, increasing the number of samples to $\left\lfloor \frac{N-(T+F)}{S} \right\rfloor + 1$. This improves data efficiency by exposing the model to multiple shifted temporal contexts.

During inference, each trajectory is first initialized using its first T observed positions. Based on this historical context, the model predicts the next F future positions. These predicted positions are then appended to the sequence, and a sliding window of length T is shifted forward by F steps so that the newly generated predictions become part of the input for the next forecasting stage. This iterative process follows an autoregressive rollout setup, allowing the model to produce long-horizon trajectory forecasts beyond a single prediction window.

3.1 Velocity-Aware State Augmentation

To enrich the model input with both position and velocity, we augment the raw position sequence with estimated velocities. Instead of estimating velocity by finite differences, which can strongly amplify measurement noise, we apply a Savitzky–Golay differentiation filter that suppresses high-frequency fluctuations while preserving the underlying motion trend.

Specifically, for each time step, we consider a short centered window of size $L = 11$, fit a local 3rd-order polynomial to the positions in this window via linear least squares, and take its derivative at the center to obtain the velocity estimate.

Let $2m + 1 = L$ (i.e., $m = 5$ for $L = 11$). This procedure can be equivalently expressed as applying a fixed linear filter over the position sequence, where the coefficients d_j are pre-computed from the polynomial fitting and differentiation process:

$$\mathbf{v}_t = \sum_{j=-m}^{m} d_j \mathbf{p}_{t+j}, \tag{1}$$

This formulation avoids repeated polynomial fitting during inference and enables efficient computation using pre-computed weights.

Both positions and estimated velocities are normalized using the maximum ℓ^2-norm computed over the training set to ensure consistent scaling across inputs:

$$\mathbf{p}_t^n = \frac{\mathbf{p}_t}{\max_k \|\mathbf{p}_k\|_2}, \quad \mathbf{v}_t^n = \frac{\mathbf{v}_t}{\max_k \|\mathbf{v}_k\|_2}. \tag{2}$$

The final augmented input to the model is:

$$\mathbf{x}_t = [\mathbf{p}_t^n, \mathbf{v}_t^n] \in \mathbb{R}^6. \tag{3}$$

3.2 GRU With Dual Output Heads

We use a GRU-based encoder–decoder architecture [4] to model trajectory dynamics. A GRU (Gated Recurrent Unit) is a recurrent neural network module that maintains a hidden state $\mathbf{h}_t$ and updates it at each time step using gating mechanisms that regulate information flow. Given an input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, the GRU computes the reset gate $\mathbf{r}_t$, update gate $\mathbf{z}_t$, new gate $\mathbf{n}_t$, and the hidden state $\mathbf{h}_t$ as follows:

$$\mathbf{r}_t = \sigma(\mathbf{W}_{ir}\mathbf{x}_t + \mathbf{b}_{ir} + \mathbf{W}_{hr}\mathbf{h}_{t-1} + \mathbf{b}_{hr}) \tag{4}$$

$$\mathbf{z}_t = \sigma(\mathbf{W}_{iz}\mathbf{x}_t + \mathbf{b}_{iz} + \mathbf{W}_{hz}\mathbf{h}_{t-1} + \mathbf{b}_{hz}) \tag{5}$$

$$\mathbf{n}_t = \tanh(\mathbf{W}_{in}\mathbf{x}_t + \mathbf{b}_{in} + \mathbf{r}_t \odot (\mathbf{W}_{hn}\mathbf{h}_{t-1} + \mathbf{b}_{hn})) \tag{6}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{n}_t + \mathbf{z}_t \odot \mathbf{h}_{t-1}, \tag{7}$$

where $\sigma(\cdot)$ denotes the sigmoid function, $\odot$ represents Hadamard product, and $\mathbf{W}_\cdot$ and $\mathbf{b}_\cdot$ are trainable weight matrices and bias vectors, respectively.

GRUs are well-suited for UAV trajectory forecasting due to their ability to model sequential dependencies in motion data. In particular, UAV trajectories exhibit both short- and longer-term temporal structure, which requires memory over multiple time steps. The gating mechanism in GRUs enables the model to selectively retain or forget historical information, making it effective for capturing such dependencies. Compared to another recurrent architecture, LSTMs (Long Short-term Memory networks), GRUs achieve comparable performance with fewer parameters, leading to more efficient training.

Based on this recurrent formulation, we construct a GRU-based encoder–decoder architecture for trajectory forecasting. The encoder is a multi-layer GRU that processes the input sequence of 6D position–velocity vectors $\{\mathbf{x}_k\}_{k=t-T+1}^{t}$ and compresses them into a latent representation, which summarizes the observed trajectory. Dropout is applied within the encoder to mitigate overfitting.

The decoder is another multi-layer GRU that is initialized with this latent representation and autoregressively generates future hidden states $\mathbf{h}_{t+k}$ for $k = 1, \ldots, F$, where each step conditions on the previously generated state.

To enable simultaneous prediction of positions and velocities, we introduce a dual-head output design. Each decoder hidden state is passed through two separate linear projections:

$$\hat{\mathbf{p}}_{t+k}^{n} = \mathbf{W}_{\text{pos}}\mathbf{h}_{t+k} + \mathbf{b}_{\text{pos}}, \quad \hat{\mathbf{v}}_{t+k}^{n} = \mathbf{W}_{\text{vel}}\mathbf{h}_{t+k} + \mathbf{b}_{\text{vel}}. \tag{8}$$

This design allows the shared recurrent backbone to learn a unified temporal representation, while the separate heads specialize in predicting spatial position and motion dynamics, improving both positional accuracy and velocity consistency.

3.3 Kinematics-Informed Learning

While the GRU-based encoder–decoder can learn trajectory patterns from data, purely data-driven prediction may lead to physically inconsistent outputs, particularly under long-horizon autoregressive rollouts. In practice, independently predicted positions and velocities can violate basic kinematic relationships. To address this issue, we incorporate kinematics-informed constraints that explicitly enforce consistency between position and velocity predictions.

Besides the standard supervised learning objective, we introduce two physics-guided constraints that encourage the predictions to satisfy the discrete-time kinematic relation:

$$\mathbf{p}_{t+1} \approx \mathbf{p}_t + \mathbf{v}_t \Delta t. \tag{9}$$

First, a velocity-integration consistency constraint reconstructs an alternative position trajectory using only predicted velocities $\hat{\mathbf{v}}_t$, starting from the last observed position $\mathbf{p}_t$:

$$\tilde{\mathbf{p}}_{t+k} = \mathbf{p}_t + \sum_{i=0}^{k-1} \hat{\mathbf{v}}_{t+i} \Delta t, \tag{10}$$

and penalizes deviations between $\hat{\mathbf{p}}_{t+k}$ and $\tilde{\mathbf{p}}_{t+k}$ across the prediction horizon. This encourages long-term consistency between integrated motion and directly predicted positions.

Second, a local kinematic constraint enforces step-wise consistency by relating consecutive predictions:

$$\dot{\mathbf{p}}_{t+k} = \hat{\mathbf{p}}_{t+k-1} + \hat{\mathbf{v}}_{t+k-1} \Delta t, \tag{11}$$

and penalizes differences between $\hat{\mathbf{p}}_{t+k}$ and $\dot{\mathbf{p}}_{t+k}$ at each step, encouraging physically plausible short-term dynamics.

These physics-guided terms are combined with standard position and velocity losses on the normalized outputs:

$$\mathcal{L}_{\text{pos}} = \text{MSE}(\hat{\mathbf{p}}_{t+k}^{n}, \mathbf{p}_{t+k}^{n}), \quad \mathcal{L}_{\text{vel}} = \text{MSE}(\hat{\mathbf{v}}_{t+k}^{n}, \mathbf{v}_{t+k}^{n}),$$
$$\mathcal{L}_{\text{con}} = \text{MSE}(\hat{\mathbf{p}}_{t+k}, \tilde{\mathbf{p}}_{t+k}), \quad \mathcal{L}_{\text{loc}} = \text{MSE}(\hat{\mathbf{p}}_{t+k}, \dot{\mathbf{p}}_{t+k}),$$

leading to the final training objective:

$$\mathcal{L} = \mathcal{L}_{\text{pos}} + \alpha\mathcal{L}_{\text{vel}} + \beta\mathcal{L}_{\text{con}} + \gamma\mathcal{L}_{\text{loc}}. \tag{12}$$

4 Experiment

4.1 Dataset

We use the UAV trajectory dataset introduced by Nacar et al. (2024), which contains both synthetic and real-world 3D flight trajectories. Specifically, the dataset combines a large synthetic set generated in Gazebo using PX4 Autopilot, where UAVs follow predefined motion patterns such as circular and infinity-like trajectories, together with real-world flight data from the public UZH-FPV and Mid-Air datasets. This combination provides diverse flight behaviors under both simulated and realistic operating conditions, improving the robustness of trajectory prediction across different patterns and environments [9].

To preserve this diversity, the trajectories are split into training, validation, and test sets using an 80/10/10 ratio, while ensuring that each subset contains both synthetic and real-world trajectories. This allows the model to be trained and evaluated on a representative mixture of simulated and more complex real-world flight dynamics.

To construct supervised training samples, we split each trajectory into multiple windows, using a history length of $T = 50$, a prediction horizon of $F = 20$, and a stride of $S = 10$.

4.2 Model Configuration

4.3 Experimental Setup

Our model uses a GRU-based encoder–decoder architecture with 2-layer GRUs. To determine an appropriate configuration, hyperparameters are tuned using grid search on the validation set. Based on this process, the final model uses a hidden dimension of 64, learning

rate 10^{-3}, and loss weights $\alpha = 0.1$, $\beta = 0.1$, and $\gamma = 0.1$, which balance the velocity and physics-guided components in the training objective.

Training is performed using the Adam optimizer for up to 200 epochs. A learning rate scheduler reduces the learning rate by a factor of 0.1 every 50 epochs to improve convergence during later training stages. Dropout with a rate of 0.5 is applied to reduce overfitting, and early stopping with a patience of 10 epochs is used based on validation performance.

To evaluate the effectiveness of the proposed model, we compare it against a diverse set of baseline architectures, including LSTM, DLinear, Informer, Pyraformer, PatchTST, NeuralODE, 3DMoTraj, and VECTOR. These baselines cover a broad range of architectures, including recurrent sequence models (LSTM), linear temporal forecasting methods (DLinear), transformer-based long-range sequence models (Informer, Pyraformer, and PatchTST), continuous-time dynamical models (NeuralODE), and trajectory-specific architectures (3DMoTraj and VECTOR). This provides a comprehensive evaluation across both general-purpose time-series forecasting models and specialized trajectory prediction methods.

4.4 Evaluation Setup

To comprehensively assess the performance of KITE against baseline methods, we use multiple metrics that capture both prediction accuracy and operational reliability of the predicted trajectories. Since UAV forecasting is not only about minimizing average error but also maintaining stable and safe over time, these complementary metrics are required.

Let $\mathbf{P} = \{\mathbf{p}_{t+k}\}_{k=1}^{K}$ denote the ground-truth trajectory and $\hat{\mathbf{P}} = \{\hat{\mathbf{p}}_{t+k}\}_{k=1}^{K}$ denote the predicted trajectory, where K is the remaining prediction horizon after time T. We define the instantaneous positional error at step k as the ℓ^2-norm distance:

$$e_k = \|\mathbf{p}_{t+k} - \hat{\mathbf{p}}_{t+k}\|_2 \tag{13}$$

We report several standard trajectory forecasting metrics to evaluate prediction quality from different perspectives:

- **Average Displacement Error (ADE):** ADE measures the mean spatial error across the entire predicted trajectory. It reflects the overall tracking accuracy and indicates how closely the predicted path follows the ground-truth path on average. A lower ADE suggests better long-term consistency.

$$\text{ADE} = \frac{1}{K}\sum_{k=1}^{K} e_k$$

- **Root Mean Square Error (RMSE):** RMSE also evaluates overall prediction accuracy, but squares the errors before averaging, making it more sensitive to large deviations. This is important in UAV applications, where occasional large prediction errors may indicate unstable motion forecasting.

$$\text{RMSE} = \sqrt{\frac{1}{K}\sum_{k=1}^{K} e_k^2}$$

- **Final Displacement Error (FDE):** FDE measures the positional error at the final predicted time step. Unlike ADE or RMSE, which averages performance over the full prediction horizon, FDE evaluates whether the model can accurately predict the UAV's final destination. This is relevant for navigation and planning tasks, where the endpoint of the predicted trajectory influences downstream control decisions.

$$\text{FDE} = e_K$$

Together, ADE and RMSE assess overall forecasting performance, while FDE focuses on long-horizon endpoint accuracy.

We also define the **valid time** T_v to measure operational reliability. This represents the maximum number of consecutive future time steps during which the predicted trajectory remains within a safety threshold δ (set to 2.0m in our experiments). If the prediction error exceeds this threshold at the first step, i.e., $e_1 \geq \delta$, then the prediction is considered unreliable from the start and $T_v = 0$. Otherwise, T_v is defined as:

$$T_v = \max\{k \in \{1, \ldots, K\} \mid e_i < \delta, \forall i \leq k\} \tag{14}$$

Unlike ADE, RMSE, and FDE, which quantify prediction error magnitude, valid time captures how long the predicted trajectory remains practically reliable before deviating beyond an acceptable tolerance. In UAV forecasting, this is particularly important because a prediction may initially be accurate but gradually drift over time. A larger T_v indicates that the model can sustain trustworthy trajectory predictions for longer horizons, making it more suitable for real-time planning, collision avoidance, and autonomous navigation.

4.5 Experimental Results

As shown in Table 1, KITE achieves the best overall performance in terms of ADE and RMSE, indicating the lowest average trajectory deviation and strongest overall tracking accuracy among all compared methods. This suggests that the proposed model is more effective at capturing general motion dynamics, leading to more accurate predictions across the entire forecasting horizon.

KITE also achieves the highest valid time T_v, demonstrating that its predictions remain within the safety threshold for the longest duration before diverging. This indicates improved temporal stability in autoregressive rollout, suggesting that the model is less prone to error accumulation over time compared to baseline models.

In terms of FDE, KITE remains highly competitive and ranks top three among the compared models. This shows that although endpoint prediction is challenging due to compounding uncertainty over long horizons, the proposed method still maintains strong long-term predictive capability and does not sacrifice endpoint accuracy for short-term performance.

Among the baselines, trajectory-specific architectures such as VECTOR and 3DMoTraj perform relatively well compared to general-purpose time-series models. VECTOR benefits from explicitly incorporating velocity information, which helps capture motion direction and improves prediction accuracy. 3DMoTraj decomposes motion along spatial axes, enabling more structured modeling of 3D dynamics and contributing to its strong performance, with the lowest FDE among compared models.

Recurrent models such as LSTM also achieve competitive results, highlighting the effectiveness of recurrent architectures for sequential trajectory modeling. This is consistent with the design of both

Model	ADE (m) ↓	RMSE (m) ↓	FDE (m) ↓	T_v (steps) ↑
LSTM [6]	1.3036 ± 0.8313	1.5540 ± 0.9399	1.8796 ± 1.3221	95.05 ± 40.64
DLinear [16]	2.6642 ± 1.0801	2.9914 ± 1.2076	2.9155 ± 1.6775	29.08 ± 29.09
Informer [19]	3.4184 ± 1.6356	3.8447 ± 1.8482	5.7801 ± 3.4938	28.50 ± 24.69
Pyraformer [8]	1.5549 ± 1.0014	1.7350 ± 1.1164	2.1728 ± 1.6503	80.42 ± 43.72
PatchTST [10]	1.7214 ± 0.6899	2.1041 ± 0.7702	2.6168 ± 1.0964	81.17 ± 40.48
NeuralODE [3]	2.8771 ± 1.3908	3.0865 ± 1.8815	2.8407 ± 1.1965	48.99 ± 26.24
3DMoTraj [18]	1.3920 ± 0.5155	1.6199 ± 0.6105	**1.4328** ± 1.0037	38.49 ± 40.38
VECTOR [9]	1.2633 ± 0.8583	1.4333 ± 0.9745	1.7322 ± 1.3639	95.59 ± 40.43
KITE	**1.2587** ± 0.7514	**1.4178** ± 0.8591	1.7554 ± 1.2283	**96.95** ± 42.60

Table 1: Performance comparison of different models (average across test trajectories).

VECTOR and 3DMoTraj, which also rely on recurrent architecture as their backbone.

In contrast, transformer-based models such as Informer, Pyraformer, and PatchTST show weaker performance in this setting. These models are primarily designed for long-range temporal correlations in general time-series data, but may struggle to capture the kinematic information in UAV motion data. DLinear is among the worst models, suggesting that purely linear models may be insufficient to model the complex and nonlinear dynamics of UAV trajectories.

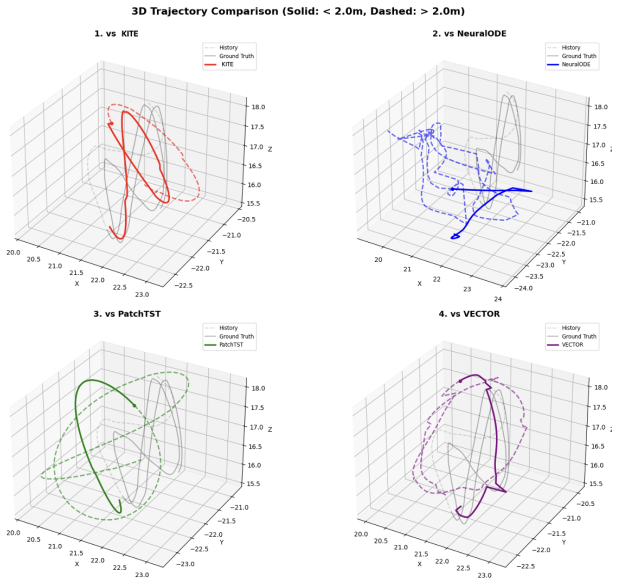


Figure 2: 3D UAV trajectory comparison between ground truth and predicted trajectories from different models

Figure 2 compares 3D UAV trajectories for KITE and three baselines on a representative test case. Each subplot shows the ground truth future trajectory in grey together with the prediction of the model (red, blue, green, or purple). The solid coloured segments indicate time steps where the prediction error is below the 2.0 m valid time threshold, while the dashed coloured segments show predictions after the trajectory has become invalid. Visually, KITE maintains a long solid segment that closely follows the ground-truth loop, whereas the baselines switch to dashed lines much earlier and drift away from the true path. Figure 3 shows the trajectory decomposed per axis, making it clear that KITE tracks the ground truth more consistently across all three coordinates than the baselines.

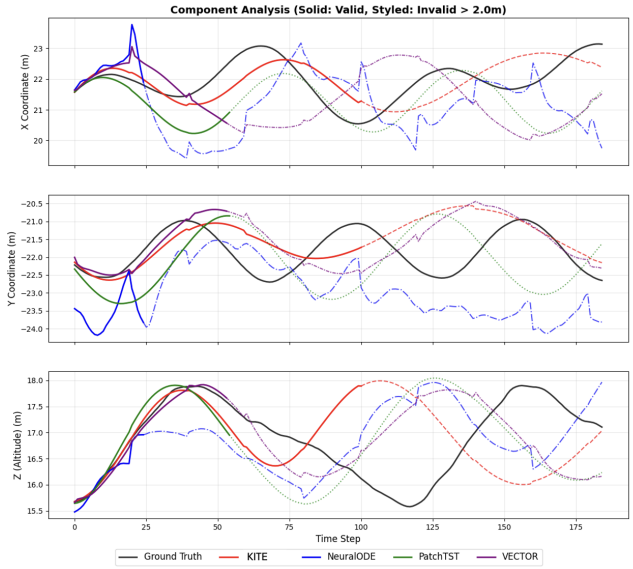


Figure 3: Per-axis comparison of ground truth and predicted UAV trajectories across different models

4.6 Ablation Study

In this section, we analyze the contribution of each key component in KITE to the overall performance. Specifically, we investigate how different design choices affect forecasting accuracy and stability, including the velocity estimation method, the input representation, and the kinematics-informed constraints. All experiments are conducted under the same training and evaluation settings as the full model.

First, we examine the impact of the velocity estimation strategy. In the full model, velocities are computed using Savitzky–Golay filtering to obtain smooth motion estimates. To evaluate its effect, we replace this step with a simple finite difference approximation, which directly computes velocity from adjacent position differences. This allows us to assess the importance of noise suppression

Variant	ADE (m) ↓	RMSE (m) ↓	FDE (m) ↓	T_v (steps) ↑
KITE	**1.2587** ± 0.7514	**1.4178** ± 0.8591	1.7554 ± 1.2283	**96.95** ± 42.60
w/o Savitzky-Golay filtering	1.4634 ± 0.8536	1.6658 ± 0.9751	2.0107 ± 1.3863	84.85 ± 39.72
w/o velocity input	2.0423 ± 0.9543	2.3842 ± 1.1240	3.0077 ± 1.9285	62.78 ± 34.89
w/o consistency constraint	1.3828 ± 0.8643	1.5633 ± 0.9889	1.9326 ± 1.3812	90.03 ± 41.95
w/o local-update constraint	1.4286 ± 0.8489	1.6197 ± 0.9719	1.9732 ± 1.3390	87.71 ± 42.43

Table 2: Performance comparison of different ablation variants

in velocity estimation. Second, we study the role of velocity in the input representation. In this variant, we remove velocity from the model input and only provide position sequences to the encoder. The model is then required to jointly infer both future positions and velocities, allowing us to evaluate whether explicitly providing motion information improves learning of dynamic behavior. Finally, we investigate the contribution of the kinematics-informed constraints, which are a key component of KITE. These constraints enforce kinematic consistency between predicted positions and velocities. To isolate their effect, we disable them by setting the corresponding loss weights to zero, i.e., $\beta = 0$ for the integration consistency constraint and $\gamma = 0$ for the local-update constraint.

As shown in Table 2, all components of KITE contribute to overall performance, with different degrees of impact on trajectory accuracy and stability. The most significant performance degradation is observed when removing velocity from the input representation, where ADE, RMSE, and FDE increase substantially and valid time decreases sharply. This indicates that integrating velocity information to the input plays an important role in capturing motion dynamics and enabling accurate trajectory forecasting.

Removing the Savitzky–Golay filtering step also leads to consistent but smaller performance drops, suggesting that velocity smoothing improves performance by reducing the effect of noise in the estimated motion signals. Removing the consistency and the local-update constraints results in reduced valid time and increased prediction error, indicating that enforcing kinematic structure helps mitigate error accumulation during autoregressive rollout. Among the two, removing the local-update constraint results in a slightly larger degradation in both trajectory accuracy and stability compared to removing the consistency constraint. This suggests that enforcing step-wise kinematic coherence plays a relatively important role in maintaining both short-term prediction accuracy and long-term rollout stability. The consistency constraint also contributes positively, but with a comparatively smaller impact on overall performance.

5 Conclusion

KITE demonstrates that embedding simple yet well-designed kinematic information into a simple GRU encoder–decoder improves 3D UAV trajectory forecasting. By augmenting positional inputs with Savitzky–Golay-smoothed velocity estimates and enforcing discrete-time kinematic consistency between predicted positions and velocities, the model achieves more accurate and physically plausible long-horizon predictions. The ablation study confirms that all components of KITE contribute to performance, though to different extents. Removing velocity from the input leads to the largest degradation, highlighting the importance of explicitly modeling motion dynamics. Among the two physics constraints, the local-update constraint has a slightly stronger effect than the consistency constraint, indicating that step-wise kinematic enforcement is particularly important for reducing error accumulation in autoregressive forecasting.

Overall, these results indicate that combining data-driven sequence modeling with simple physics-inspired inductive biases leads to substantial gains in both accuracy and temporal stability. This makes KITE a practical and reliable approach for real-world applications such as online trajectory planning, autonomous navigation, and safe UAV operation.

References

[1] Márcio da Silva Arantes, Jesimar da Silva Arantes, Claudio Fabiano Motta Toledo, and Brian C Williams. 2016. A hybrid multi-population genetic algorithm for UAV path planning. In *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. 853–860.

[2] Samet Ayhan and Hanan Samet. 2016. Aircraft trajectory prediction made easy with predictive analytics. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 21–30.

[3] Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. 2018. Neural ordinary differential equations. *Advances in neural information processing systems* 31 (2018).

[4] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Alessandro Moschitti, Bo Pang, and Walter Daelemans (Eds.). Association for Computational Linguistics, Doha, Qatar, 1724–1734.

[5] Weibin Gu, Stefano Primatesta, and Alessandro Rizzo. 2024. Physics-informed neural network for quadrotor dynamical modeling. *Robotics and Autonomous Systems* 171 (2024), 104569.

[6] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Comput.* 9, 8 (Nov. 1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735

[7] Peiyan Jia, Huiping Chen, Lei Zhang, and Daojun Han. 2022. Attention-LSTM based prediction model for aircraft 4-D trajectory. *Scientific reports* 12, 1 (2022), 15533.

[8] Shizhan Liu, Hang Yu, Cong Liao, Jianguo Li, Weiyao Lin, Alex X. Liu, and Schahram Dustdar. 2022. Pyraformer: Low-Complexity Pyramidal Attention for Long-Range Time Series Modeling and Forecasting. In *International Conference on Learning Representations*.

[9] Omer Nacar, Mohamed Abdelkader, Lahouari Ghouti, Khaled Gabr, Abdulrahman Al-Batati, and Anis Koubaa. 2024. VECTOR: Velocity-Enhanced GRU Neural Network for Real-Time 3D UAV Trajectory Prediction. *Drones* 9, 1 (Dec. 2024), 8. doi:10.3390/drones9010008

[10] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.

[11] Gil Serrano, Marcelo Jacinto, José Ribeiro-Gomes, João Pinto, Bruno J Guerreiro, Alexandre Bernardino, and Rita Cunha. 2024. Physics-Informed Neural Network for Multirotor Slung Load Systems Modeling. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 12592–12598.

[12] Pushpak Shukla, Shailendra Shukla, and Amit Kumar Singh. 2024. Trajectory-prediction techniques for unmanned aerial vehicles (UAVs): A comprehensive survey. *IEEE Communications Surveys & Tutorials* 27, 3 (2024), 1867–1910.

[13] Jorge Silvestre, Paula Mielgo, Anibal Bregon, Miguel A Martínez-Prieto, and Pedro C Álvarez-Esteban. 2024. Towards aircraft trajectory prediction using LSTM networks. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 1059–1060.

[14] Zhengyi Wang, Man Liang, and Daniel Delahaye. 2017. Short-term 4d trajectory prediction using machine learning methods. In *SID 2017, 7th SESAR Innovation Days*.

[15] Zhanji Yang, Xiaolei Kang, Yuanhao Gong, and Jiansheng Wang. 2023. Aircraft trajectory prediction and aviation safety in ADS-B failure conditions based on neural network. *Scientific reports* 13, 1 (2023), 19677.

[16] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1248, 8 pages.

[17] Yifan Zhang, Ziye Jia, Chao Dong, Yuntian Liu, Lei Zhang, and Qihui Wu. 2022. Recurrent LSTM-based UAV trajectory prediction with ADS-B information. In *GLOBECOM 2022-2022 IEEE Global Communications Conference*. IEEE, 1–6.

[18] Hao Zhou, Xu Yang, Mingyu Fan, Lu Qi, Xiangtai Li, Ming-Hsuan Yang, and Fei Luo. 2025. Three-Dimensional Trajectory Prediction with 3DMoTraj Dataset. In *Forty-second International Conference on Machine Learning*.

[19] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Conference*, Vol. 35. AAAI Press, 11106–11115.

[20] Yian Zhu, Ziye Jia, Qihui Wu, Chao Dong, Zirui Zhuang, Huiling Hu, and Qi Cai. 2024. UAV trajectory tracking via RNN-enhanced IMM-KF with ADS-B data. In *2024 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 1–6.